

SHOM – Service Hydrographique et Océanographique de la Marine

Architecture de la chaîne ArchFCU

Proposition de mise en place et justification des choix techniques

Rédigé par : Thomas Graber ; Avec la contribution
(conception de la chaîne) de Didier Gonthier
13/06/2025

Table des matières

I.	Contextes et objectifs.....	5
II.	Vue d'ensemble de l'architecture.....	6
III.	Descriptions des composants	7
1.	Serveur ArchFCU	7
2.	UploadWebApp.....	8
3.	Fossil SCM – Server	11
4.	Base SQLite – Journalisation des actions.....	11
5.	Serveur Jenkins (Ediacara)	14
IV.	Choix technique et technologique.	15
6.	Choix de fossil comme logiciel de suivi de version des cartes.....	15
7.	Choix de SQLite pour la journalisation	16

Table des figures

Figure 1 Diagramme d'architecture de la chaine archFCU.....	6
Figure 2 Arborescence de fichier des données du serveur ArchFCU.....	8
Figure 3 Processus d'upload d'une nouvelle carte FCU	11
Figure 4 Résultat du test de performance de SQLite: Nombre d'échec en fonction du nombre de clients	17
Figure 5 Résultat du test de performance de SQLite: Débit (Writes/s) en fonction du nombre de clients	18

Glossaire

BDMR	Dépôt SVN servant de base de données pour les cartes papiers de la chaîne actuelle de production.
EDIACARA	Serveur Windows réalisant grâce à Jenkins des tests et la diffusion des cartes papiers du SHOM.
<u>EIFFEL</u>	Langage de programmation compilé, utilisé par la chaîne de production des cartes papiers. Le langage est puissant mais plus maintenu.
FCU	« Fond de carte unifiée ». Nouveau programme de production de cartes papiers.
<u>FOSSIL</u>	Logiciel de gestion de versions distribué, basé sur SQLite.
GRASTER	Logiciel développé en interne, utilisé par les cartographes dans leur travail pour prendre en charge une carte et la publier.
<u>HTTP</u>	HyperText Transfer Protocol. Protocole utilisé pour les échanges de données sur le Web.
IHM	Interface Homme-Machine. Interface utilisateur de l'application web (ex. UploadWebApp).
<u>JENKINS</u>	Outil d'intégration continue et d'automatisation.
<u>JSON</u>	JavaScript Object Notation. Format léger d'échange de données.
TIFF, PNG, PDF, XML	Formats de fichiers.
<u>PHP</u>	Langage de script côté serveur, utilisé pour développer des interfaces web front-end.
<u>PYTHON</u>	Langage de programmation interprété.
<u>SAMBA</u>	Logiciel libre permettant le partage de fichiers entre systèmes Linux et Windows via SMB/CIFS.
SQLITE	Base de données relationnelle embarquée (un seul fichier) de type SQL.
SSH	<i>Secure Shell</i> . Protocole sécurisé d'accès distant aux serveurs (administration).

I. Contextes et objectifs

La nouvelle chaîne des cartes papiers « Fond de Cartes Unifié » (FCU) est en cours de développement au SHOM.

Les premières cartes sont prêtes à être produites, et les outils existants — notamment Shom Produit — ont déjà été en grande partie adaptés pour pouvoir les intégrer.

Il devient maintenant nécessaire de mettre en place une solution simple et efficace pour archiver et diffuser ces cartes, qui sont identifiées par un numéro d'édition et un numéro de correction.

Le système de stockage actuel (BDMR), les outils de travail et de dépôt (Graster), ainsi que la chaîne de traitement et de diffusion existante (Ediacara/Jenkins) ne sont pas adaptés à ces nouvelles cartes. En effet ils reposent sur du code écrit en Eiffel, peu facilement modifiable, de par sa complexité et son manque de documentation. Cette chaîne complète a vocation à évoluer, mais ce ne sera pas pour tout de suite.

Le choix a donc été fait de développer une nouvelle chaîne dédiée, plus légère, plus pratique, spécifiquement pensée pour l'archivage et la diffusion des cartes FCU.

Les objectifs relevés sont les suivants :

- Pouvoir archiver les nouvelles versions de cartes au moyen d'une IHM.
- Pouvoir retrouver les cartes d'une édition et corrections de carte simplement.
- Pouvoir retrouver l'historique de toutes les actions au niveau du dépôt global.
- Limiter au maximum la complexité en infrastructure.

Ce document a pour objectif de présenter cette proposition de chaîne, avec son architecture technique (physique et logicielle), ainsi que son fonctionnement général.

II. Vue d'ensemble de l'architecture

Cette section décrit une proposition d'architecture pour la nouvelle chaîne d'archivage et de diffusion des cartes du FCU.

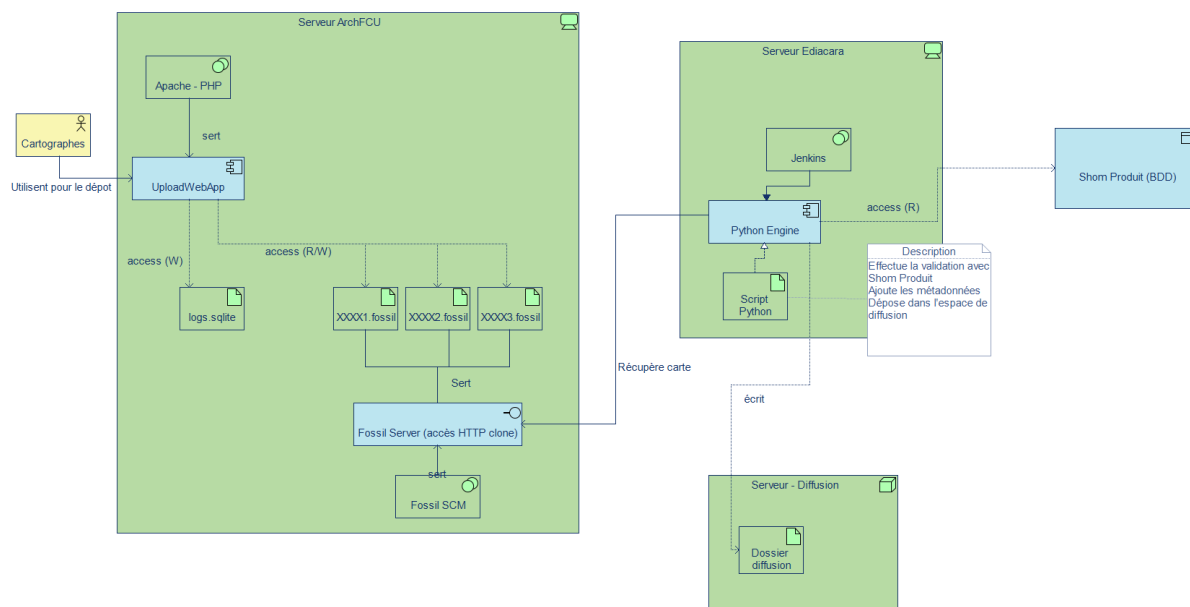


Figure 1 Diagramme d'architecture de la chaîne archFCU

Les cartographes utilisent une application web en glisser-déposer pour proposer une nouvelle version d'une carte. Cette interface simple et dédiée est hébergée sur un nouveau serveur Linux nommé ArchFCU.

Chaque action réalisée via cette interface (téléversement, remplacement, validation) est automatiquement journalisée dans une base SQLite, utilisée comme mécanisme de traçabilité léger.

Les fichiers déposés sont intégrés à des dépôts Fossil, permettant un suivi de version précis de chaque carte. Un serveur HTTP Fossil peut être lancé ponctuellement pour rendre les dépôts accessibles à d'autres outils, si nécessaire.

Le serveur Ediacara, déjà en service dans la chaîne de production papier existante, est réutilisé pour la phase de traitement et de diffusion. Un job Jenkins récupère les dépôts, valide leur cohérence via la base métier Shom Produit, ajoute les métadonnées requises, puis dépose les cartes dans l'espace de diffusion du SHOM.

La section suivante détaille chaque composant de cette chaîne. Les choix techniques (SQLite, Fossil, structure des scripts, etc.) sont justifiés plus loin dans le document, dans la section dédiée aux choix d'architecture.

III. Descriptions des composants

1. Serveur ArchFCU

Le serveur ArchFCU est l'élément central de la nouvelle chaîne FCU. Il héberge à la fois l'interface de dépôt utilisée par les cartographes et l'ensemble des dépôts Fossil correspondant aux cartes. Il propose également un point d'accès HTTP pour exposer les dépôts à d'éventuels outils tiers.

Caractéristiques générales :

- Système d'exploitation : Linux
- Services hébergés :
 - Apache + PHP : héberge l'application web de dépôt
 - Fossil SCM : gestion des dépôts versionnés (installé sous forme d'exécutable)
 - Base de données locale : logs.sqlite, utilisée pour la journalisation
- Accès réseau :
 - Port 80 : pour l'accès à l'interface web de dépôt
 - Port 8080 : pour l'exposition HTTP des dépôts Fossil (serveur Fossil lancé à la demande)
 - Port 22 (SSH) : pour l'administration système

Organisation des dépôts :

- Un dépôt Fossil dédié est créé pour chaque carte.
- Les dépôts sont conservés localement sous forme de fichiers « .fossil »

Sécurité :

- Le serveur est uniquement accessible depuis le réseau interne du SHOM.
- L'interface de dépôt n'est pas exposée à l'extérieur.
- Des droits restreints sur les répertoires de dépôts et sur le fichier logs.sqlite permettent de contenir les accès non désirés.

Note : Le serveur n'a pas besoin d'accès Internet, toutes les opérations sont réalisées en local ou en réseau interne.

Proposition d'architecture des dépôts :

- Un dossier « Dépôt » avec :
 - Un dossier « en service » avec :
 - Un dossier « NP » qui contient les fichiers fossil de chaque carte non protégée et en service
 - Un dossier « DR » qui contient les fichiers fossil de chaque carte protégée et en service
 - Un dossier « supprimées » avec :
 - Un dossier « NP » qui contient les fichiers fossil de chaque carte non protégée et hors-service
 - Un dossier « DR » qui contient les fichiers fossil de chaque carte protégée et hors-service
- Un dossier « Journal » avec :
 - Un fichier « logs.sqlite », base de données au format SQLite.

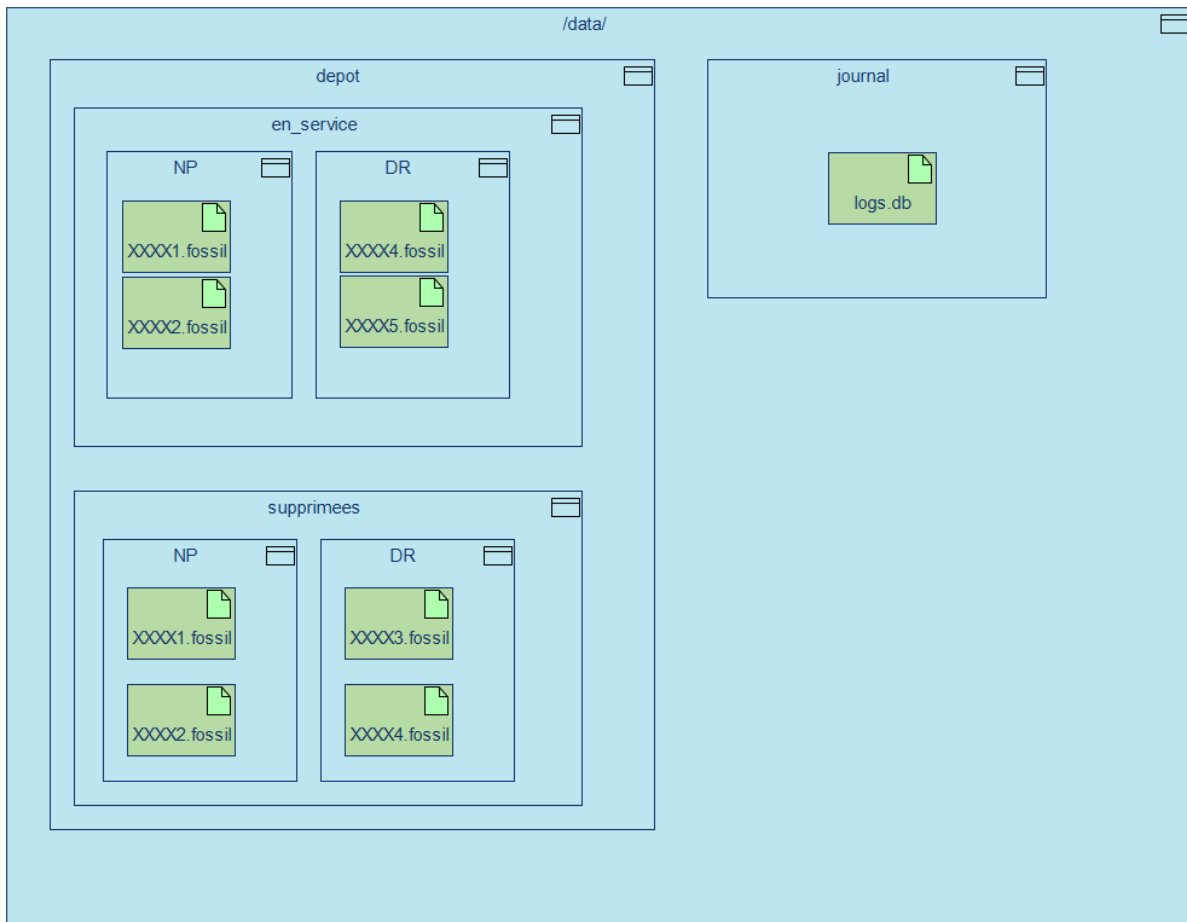


Figure 2 Arborescence de fichier des données du serveur ArchFCU

2. UploadWebApp

L'UploadWebApp est l'interface web dédiée aux cartographes pour le dépôt de nouvelles versions de cartes papier. Elle est conçue pour être simple d'utilisation, en glisser-déposer, et suffisamment robuste pour garantir la traçabilité de toutes les opérations.

Cette application web est développée en PHP et hébergée sur le serveur ArchFCU, via Apache.

Fonctionnement général :

- Lorsqu'un utilisateur se connecte, l'interface vérifie la présence de son fichier de configuration JSON.
- Ce fichier détermine ses droits d'accès (lecture, écriture, dépôt, etc.).
- Si le fichier n'existe pas, l'accès est bloqué.
- Après authentification :
 - L'utilisateur glisse un dossier contenant les fichiers d'une nouvelle version d'une carte.
 - Des validations sont effectuées (nombre de fichiers, cohérence, formats, etc)
 - Les nouveaux fichiers sont téléversés (TIFF, XML, PDF, PNG) dans le dépôt fossil relatif à la carte, et le crée si besoin. L'application ouvre le dépôt Fossil correspondant en local, remplace les fichiers, effectue un commit, puis journalise l'opération dans la base logs.sqlite.

- Architecture des fichiers associés :
 - Chaque utilisateur dispose d'un fichier username.json dans un dossier de configuration.
 - Ce fichier contient les droits de l'utilisateur
 - L'application PHP vérifie ce fichier avant toute opération.

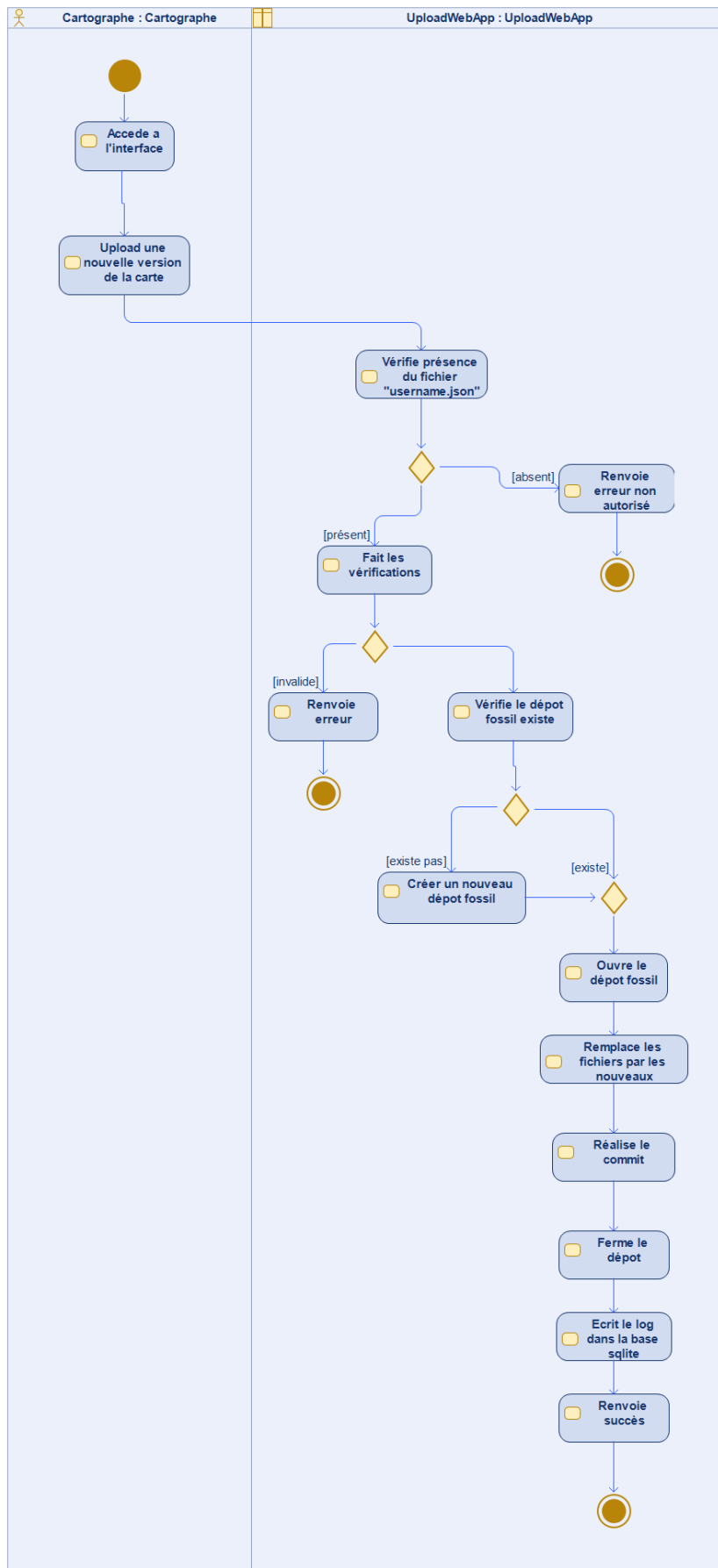
Concurrence et fiabilité :

- L'écriture dans la base SQLite est protégée par un mécanisme de retry automatique :
 - Jusqu'à 5 tentatives (200ms).

Permet de gérer les accès concurrents sans blocage, tout en restant simple et léger.

Sécurité :

- L'accès à l'application est restreint au réseau interne.
- Aucun dépôt en anonyme n'est autorisé
- Le contenu des fichiers JSON n'est pas modifiable via l'interface.



3. Fossil SCM – Server

La gestion des versions de cartes repose sur Fossil SCM, un outil léger et autonome de gestion de versions, adapté à un usage en local sans dépendance réseau externe. Chaque carte est représentée par un dépôt Fossil individuel, stocké localement sous forme de fichier .fossil.

Fossil est utilisé de deux façons distinctes :

- En mode ligne de commande, pour toutes les opérations d'écriture automatisées (remplacement de fichiers, commit, etc.)
- En mode serveur http pour les accès à distance.

Fonctionnement :

Lorsqu'un cartographe dépose une nouvelle version via l'UploadWebApp :

- L'application ouvre localement le dépôt Fossil associé à la carte.
- Les fichiers sont remplacés dans le dépôt.
- Un commit est effectué automatiquement avec un commentaire généré.

L'historique des versions est donc conservé intégralement et de manière indépendante pour chaque carte.

Proposition de nom de commit :

Les commits pourraient être nommés par :

« XX_YY ».

Avec XX le numéro de l'édition et YY le numéro de la correction.

Serveur HTTP Fossil :

Un serveur HTTP peut être lancé sur le port 8080.

Il permet aux outils internes de cloner ou consulter un dépôt Fossil via un navigateur ou un client Fossil.

Organisation des dépôts :

Tous les fichiers « fossil » sont stockés dans une arborescence structurée (voir section précédentes).

La distinction entre cartes en service / supprimées, protégées / non protégées est gérée par dossier.

Sécurité :

- Aucun dépôt n'est modifiable autrement que via l'interface UploadWebApp.
- L'accès en lecture via HTTP est uniquement possible depuis le réseau interne, et en lecture seule.

4. Base SQLite – Journalisation des actions

La traçabilité des dépôts est assurée par une base de données légère au format SQLite, stockée localement sur le serveur ArchFCU. Cette base, nommée « logs.sqlite », enregistre toutes les actions réalisées via l'interface UploadWebApp.

Elle constitue une preuve d'action pour chaque dépôt, et peut être consultée en cas de vérification et de problèmes.

Fonctionnement :

Chaque opération effectuée par un cartographe (téléversement, commit, tentative échouée, etc.) donne lieu à un enregistrement dans la base.

Les données enregistrées incluent :

- Identifiant de l'utilisateur
- Carte concernée
- Numéro d'édition / correction
- Date et heure
- Statut de l'opération
- Message d'erreur éventuel

Gestion de la concurrence :

SQLite ne permet qu'un écrivain à la fois.

Pour limiter les risques de blocage, l'application PHP utilise un mécanisme de retry automatique

- Paramétrage de SQLite pour attendre 5 secondes avant d'envoyer une erreur de verrou.
- Jusqu'à 5 tentatives espacées de 200 ms.
- En cas d'échec final, un message est affiché à l'utilisateur, l'invitant à ressayer.
- La base est configurée en Write-Ahead Logging (WAL) :
 - Améliore la performance en lecture simultanée
 - Permet une meilleure robustesse en cas d'accès concurrents modérés.

Emplacement :

Le fichier logs.sqlite est stocké dans un dossier séparé :

/journal/logs.sqlite

Il est protégé en écriture :

Seule l'application UploadWebApp peut y écrire.

Aucune écriture directe manuelle n'est autorisée, sauf pour SYSPROD.

Sécurité :

Permissions restrictives sur le fichier .sqlite.

Limites :

- Ce choix reste pertinent tant que le volume d'accès simultané reste faible, ce qui est attendu dans ce contexte.
- Une migration vers un moteur plus robuste (ex. PostgreSQL) pourrait être envisagée si l'usage évolue.

5. Serveur Jenkins (Ediacara)

Le serveur Ediacara est un serveur existant déjà utilisé dans la chaîne de production papier actuelle du SHOM. Il est ici réutilisé pour assurer les étapes de traitement et de diffusion des cartes issues de la nouvelle chaîne FCU.

Il joue un rôle essentiel dans la transition entre le dépôt et la diffusion, en automatisant les vérifications, les enrichissements et la mise à disposition finale.

Fonctionnement général :

Un job Jenkins est planifié (ou déclenché à la demande) pour :

- Récupérer les nouveaux dépôts Fossil depuis ArchFCU.
- Valider la cohérence des données déposées.
- Interroger la base métier Shom Produit pour compléter les métadonnées.
- Générer les produits finaux et les déposer dans le répertoire de diffusion.

Emplacement de diffusion :

Les cartes finales sont déposées dans le dossier DEPOTS_FCU du montage DATA_CARTO. Le dossier doit être créé préalablement.

Les modalités doivent encore être définies.

Connexion à ArchFCU :

Le serveur Ediacara accède aux dépôts de carte via HTTP par le serveur fossil mis en place sur le serveur.

Il accède au logs SQLite par montage samba ou par HTTP (reste à définir).

Outils utilisés :

- Jenkins : orchestrateur de tâches automatisées.
- Script Python ou bash internes : traitement, validation, conversion si besoin.

IV. Choix technique et technologique.

6. Choix de fossil comme logiciel de suivi de version des cartes

Dans la chaîne actuelle, toutes les cartes papier sont regroupées dans un unique dépôt SVN, avec tout leur historique. Ce fonctionnement, bien qu'ayant répondu aux besoins initiaux, montre aujourd'hui ses limites techniques :

- La taille du dépôt SVN devient de plus en plus importante au fil du temps.
- Les temps d'accès s'allongent à mesure que l'historique croît.

La structure unique rend impossible la suppression propre d'une carte : même une carte hors-service reste présente dans l'historique complet.

Pourquoi Fossil ?

Fossil SCM est un gestionnaire de version distribué, comparable à Git dans ses usages, mais avec plusieurs avantages spécifiques :

- Il est autonome, livré sous forme d'un simple exécutable (~10 Mo).
- Chaque dépôt est un fichier SQLite unique, contenant :
 - o Toutes les versions de fichiers,
 - o Les commentaires, commits, tags,
 - o Les utilisateurs,
 - o Et même une documentation embarquée.

Fossil intègre également un serveur HTTP embarqué, permettant de consulter un dépôt (log, timeline, fichiers, etc.) depuis un simple navigateur, sans configuration externe.

Avantages de Fossil pour notre usage

- Un dépôt par carte : on isole chaque carte dans son propre dépôt fossil, ce qui :
 - o Réduit drastiquement les temps d'accès,
 - o Permet de supprimer proprement une carte en déplaçant simplement son fichier de dépôt
- Simplicité de déploiement :
 - o Aucun logiciel à installer : un exécutable suffit, côté serveur comme côté client.
 - o Pas besoin de serveur complexe, ni de base centralisée.
- Bonne gestion des fichiers binaires :
 - o Fossil compresse efficacement les fichiers lourds (TIFF, PDF),
 - o Et ne souffre pas des limitations classiques de Git avec les binaires
- Consultation réseau facilitée :
 - o Le serveur HTTP intégré permet de partager les dépôts en lecture sur le réseau interne via un simple port.
- Fiabilité et traçabilité :
 - o Tous les commits sont horodatés, signés, et disponibles via une interface claire.

Note : La légèreté de Fossil et son intégration avec SQLite s'alignent parfaitement avec l'esprit de cette nouvelle chaîne : une solution simple, autonome, locale et robuste, sans dépendance à une architecture lourde ou à un service centralisé.

Son adoption reste entièrement compatible avec une évolution future vers une solution plus complète si le besoin émerge.

7. Choix de SQLite pour la journalisation

Le choix d'utiliser SQLite pour enregistrer les journaux d'activité (logs de dépôt) repose sur plusieurs critères : simplicité de mise en œuvre, légèreté, absence de dépendance externe et bonne robustesse en lecture/écriture, tant que le système est bien conçu.

- **Pas de serveur à déployer** : la base est un simple fichier local (logs.sqlite), sans processus serveur.
- **Intégration directe avec PHP** : permet un accès natif depuis l'application sans surcouche.
- **Lecture et écriture rapides**, surtout sur des volumes modérés comme ceux attendus ici.
- **Sécurité maîtrisée** : le fichier reste en local, avec des droits stricts.

Concurrence maîtrisée : pas un problème dans notre cas

Le fait que SQLite est bloquant en écriture simultanée nous inquiétait au début. Cependant, après quelques tests et recherches, on peut limiter le problème en gérant bien le côté applicatif.

Dans notre cas, la gestion est assurée directement dans le code PHP via :

- Des transactions courtes, une opération = une transaction,
- Un mécanisme de retry en cas de verrou (database is locked) maximum 5 fois
- Activation du mode WAL (Write-Ahead Logging) pour permettre des lectures pendant les écritures.

L'article SQLite «<https://tenthousandmeters.com/blog/sqlite-concurrent-writes-and-database-is-locked-errors/>» démontre qu'avec la bonne configuration :

- PRAGMA journal_mode = WAL;
- PRAGMA synchronous = NORMAL;
- PRAGMA busy_timeout = 5000;

... l'auteur parvient à gérer jusqu'à 128 processus en parallèle, totalisant plus de 62 000 écritures par seconde sur la base. Ce niveau de charge est largement supérieur à ce qui est attendu dans notre cas, même dans les scénarios les plus pessimistes.

Tests internes réalisés

Des tests réalisés en interne (via un script Python multi-processus) ont permis de confirmer les bons résultats dans notre environnement.

Avec 5 essais, chaque processus réalise 100 opérations d'écritures par secondes. Ce scénario a été testé avec des nombres différents de processus : [1, 2, 5, 10, 20, 50, 100].

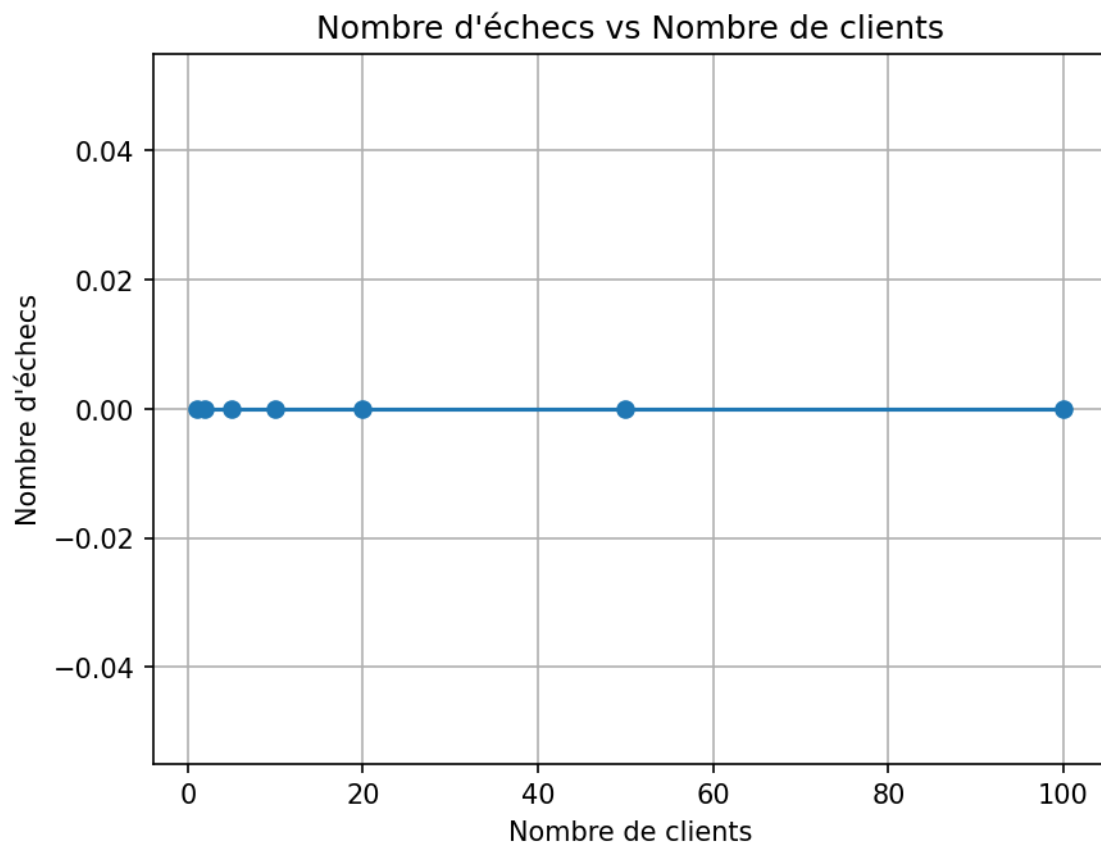


Figure 4 Résultat du test de performance de SQLite: Nombre d'échec en fonction du nombre de clients

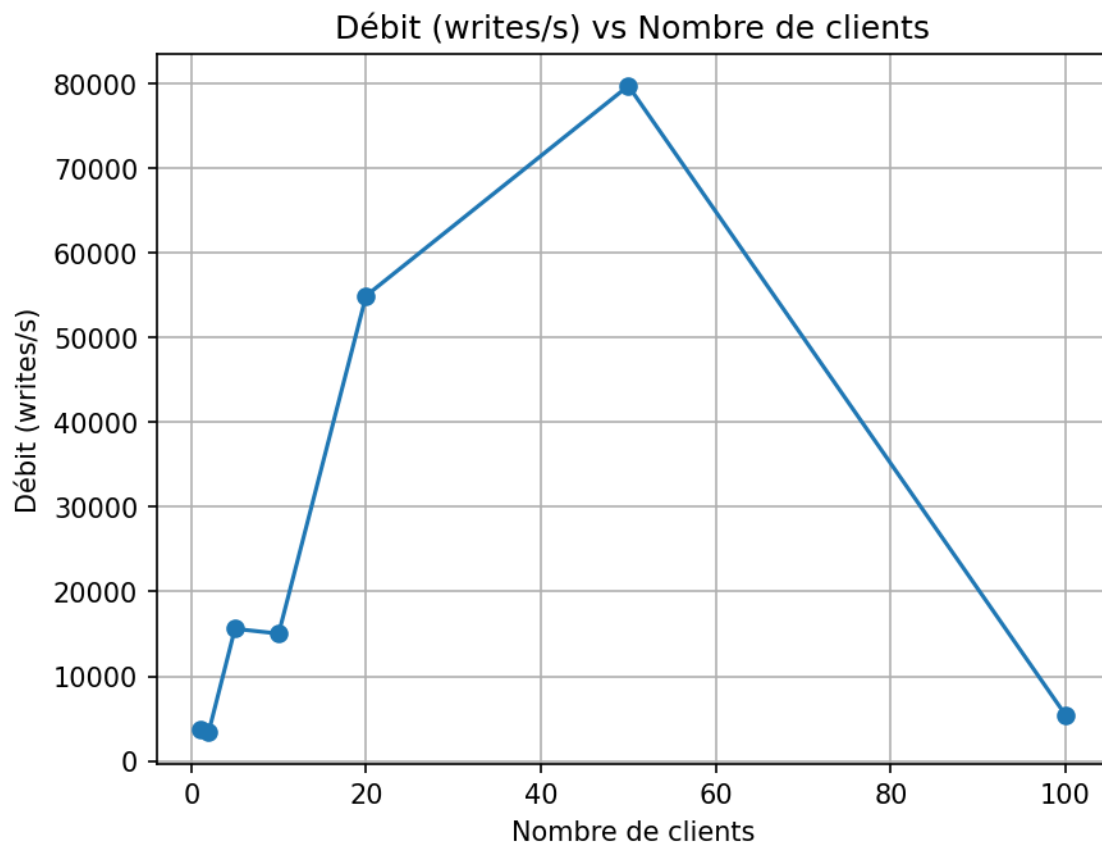


Figure 5 Résultat du test de performance de SQLite: Débit (Writes/s) en fonction du nombre de clients

Ces tests confirment que pour un usage modéré avec quelques dépôts par jour, la robustesse de SQLite est suffisante sans recourir à une base plus lourde (comme PostgreSQL).

SQLite reste un choix parfaitement cohérent pour une architecture locale, mono-serveur, avec peu de concurrence réelle.